

Paper Empire

Codex Schrittplan

Desktop/Steam-first - Godot 4 - GDScript

Stand: 2026-05-09

Zweck

Dieser Schrittplan ersetzt den alten Electron/Web-Schrittplan. Paper Empire wird ab jetzt als Desktop/Steam-first-Spiel mit Godot 4 geplant. Codex soll immer nur einen einzelnen Step umsetzen, damit das Projekt nach jedem Schritt lauffaehig und pruefbar bleibt.

0. Fixe technische Entscheidung

Bereich	Festlegung
Zielpattform	Windows Desktop zuerst; Steam spaeter. Keine Webversion und keine Mobile-Version im MVP.
Engine	Godot 4.x, 2D-Projekt.
Sprache	GDScript mit statischer Typisierung, class_name und klaren Script-Verantwortlichkeiten.
Karte	WorldMap mit CountryNodes. Laender sind klickbare Area2D/Polygon2D-Objekte mit vereinfachten Shapes im Prototyp.
Simulation	Fester Simulationstick fuer Spielregeln; visuelle Animationen getrennt ueber _process(), Tween, AnimationPlayer und Partikel.
UI	Godot Control Nodes fuer Topbar, Panels, Modals, Shop, Army Editor, Settings.
Effekte	Tweens, AnimationPlayer, GPUParticles2D/CPUParticles2D, AudioStreamPlayer.
Save	Lokale Save-Dateien unter user:// mit schemaVersion. Steam Cloud erst spaeter vorbereiten.
Steam	Spaeter ueber PlatformService und optional GodotSteam. Keine echte Steamworks-Integration vor stabilem Vertical Slice.
Codex	VSCode + Codex fuer Scripts, Daten und kleine Szenen-Anpassungen. Godot Editor bleibt fuer Szenenstruktur autoritativ.

1. Globale Codex-Regeln

- Arbeite immer nur den genannten Step ab. Keine Vorwegnahme spaeterer Phasen.
- Codex darf .tscn-Dateien nur aendern, wenn der Step es ausdruuecklich verlangt.
- Keine Umbenennung existierender Nodes ohne ausdruuecklichen Auftrag.
- Keine Game Logic in UI-Control-Scripts. UI ruft Commands/Services auf und zeigt State an.
- Keine visuellen Effekte in Simulationslogik. Simulation erzeugt Events; EffectsLayer konsumiert Events.
- Keine Steam-Abhaengigkeit im Core. Steam wird spaeter ueber ein Interface gekapselt.
- Keine Web-, Electron-, React-, TypeScript-, Browser- oder Mobile-Architektur mehr hinzufuegen.
- GDScript moeglichst statisch typisieren: Variablen, Rueckgabewerte und Funktionsparameter typisieren.
- Nach jedem Step: Projekt in Godot oeffnen/starten. Falls Godot CLI verfuegbar ist: godot --headless --path . --quit pruefen.
- Nach jedem Step: geaenderte Dateien, manuelle Checks und offene TODOs knapp dokumentieren.

2. Globale Definition of Done

- Das Godot-Projekt oeffnet ohne Parser- oder Importfehler.
- Main.tscn startet und zeigt keine Blocker-Fehler im Output.
- Neue Scripts haben klare Verantwortung und keine zufaelligen Cross-Dependencies.
- Neue Daten werden validiert oder bewusst als Prototype Fixture markiert.
- Ein Step endet nicht mit halb verbundenen Signalen, fehlenden Node-Pfaden oder kaputten Szenenreferenzen.
- Keine Future-Features im MVP: Diplomatie, Marine, Handel, Stabilitaet, Provinzen, Forschung, Multiplayer.

3. Empfohlener Codex-Arbeitsmodus

- Vor jedem Step einen Git-Checkpoint machen.
- Codex mit exakter Step-ID beauftragen: "Implement Step 7.3 only".
- Codex soll am Ende eine kurze Aenderungsliste liefern.
- Szenenarbeit: erst Struktur im Godot Editor anlegen, dann Codex Scripts schreiben lassen. Nur kleine .tscn-Patches erlauben.
- Bei Unsicherheit soll Codex einen kleinen Stub/TODO setzen, nicht ein grosses System erfinden.

4. Standard-Prompt fuer jeden Step

```
Implement Step X.Y only for Paper Empire.
This is a Godot 4 desktop/Steam-first project.
There is no web version, no Electron version, no React version and no mobile version.
Follow AGENTS.md and the step plan exactly.
Do not implement later phases.
Do not rename existing nodes unless this step explicitly requires it.
Use typed GDScript where practical.
If you edit scenes, keep the changes minimal and explain them.
```

Afterwards, report changed files, checks performed and remaining TODOs.

5. Einzelne Entwicklungsphasen und Codex-Steps

Arbeitsregel fuer alle Phasen

Nach jeder Phase gilt ein Gate: Nicht weitergehen, wenn Main.tscn nicht startet, Parserfehler im Output stehen oder der vorherige Step halb fertig ist.

Phase 1. Projektbasis und Repository

Step 1.1 - Godot Projekt erstellen

Codex-Aufgabe	Erstelle ein leeres Godot-4-2D-Projekt mit dem Namen Paper Empire und einer Main.tscn als Startszene.
Nicht tun	Keine Spielmechanik, keine Karte, keine UI-Systeme.
Definition of Done	Projekt oeffnet in Godot. Main.tscn ist als Main Scene gesetzt.
Checks	Godot Editor oeffnen; Main.tscn starten.

Step 1.2 - Git und Ignore-Dateien

Codex-Aufgabe	Richte Git-Grundlage und eine passende .gitignore fuer Godot ein.
Nicht tun	Keine Binaerdateien oder Import-Caches absichtlich committen.
Definition of Done	Repo ist sauber. Godot .import/.godot Cache-Dateien werden sinnvoll ignoriert.
Checks	git status pruefen.

Step 1.3 - AGENTS.md fuer Codex

Codex-Aufgabe	Erstelle AGENTS.md mit Godot-4-Regeln, Architekturgrenzen, Codex-Verhalten und Checks.
Nicht tun	Keine Implementierung von Gameplay.
Definition of Done	AGENTS.md sagt klar: Desktop/Steam-first, Godot 4, GDScript, keine Webversion.
Checks	Datei manuell pruefen.

Step 1.4 - Basisordner anlegen

Codex-Aufgabe	Lege res://scenes, scripts, data, ui, map, effects, audio, save, platform, tests_debug und docs an.
Nicht tun	Keine ungenutzten Frameworks oder Addons.
Definition of Done	Ordnerstruktur existiert und ist knapp dokumentiert.
Checks	Projekt neu laden, keine Importfehler.

Step 1.5 - Main Scene Skeleton

Codex-Aufgabe	Erzeuge in Main.tscn eine klare Root-Struktur mit GameRoot, WorldRoot, UIRoot, Managers.
Nicht tun	Noch keine echten Manager-Systeme implementieren.
Definition of Done	Node-Struktur ist stabil und wird im Dokument/README kurz beschrieben.
Checks	Main.tscn starten.

Step 1.6 - Projekt-README

Codex-Aufgabe	Erstelle README.md mit Startanleitung, Godot-Version, Codex-Workflow und MVP-Nichtzielen.
Nicht tun	Keine Marketing-Seite schreiben.
Definition of Done	Neue Entwickler verstehen, wie das Projekt geoeffnet und gestartet wird.
Checks	README manuell pruefen.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 2. Godot Coding- und Szenenstandards

Step 2.1 - Namenskonventionen

Codex-Aufgabe	Dokumentiere Node-, Script-, Signal- und Dateinamen-Konventionen fuer das Projekt.
Nicht tun	Keine bestehende Szenenstruktur gross umbauen.
Definition of Done	Konventionen stehen in docs/coding_standards.md.
Checks	Manuelle Pruefung.

Step 2.2 - Script-Basisklassen

Codex-Aufgabe	Lege minimale Basisscripts fuer GameManager, SimulationManager, EventBus, AudioManager und SaveManager als Stubs an.
Nicht tun	Keine echte Logik einbauen.
Definition of Done	Stubs sind typisiert, starten ohne Fehler und haben klare TODOs.
Checks	Main.tscn starten, Output pruefen.

Step 2.3 - Signal-Regeln

Codex-Aufgabe	Definiere, welche Systeme ueber Signale kommunizieren: UI -> Commands, Simulation -> GameEvents, Effects -> EventBus.
Nicht tun	Keine komplexe Event-Architektur bauen.
Definition of Done	Regeln sind dokumentiert und Stubs passen dazu.
Checks	Manuelle Pruefung.

Step 2.4 - Headless/Smoke Check

Codex-Aufgabe	Fuege eine kurze docs/checks.md mit lokalen Pruefbefehlen und manueller Checkliste hinzu.
Nicht tun	Keine CI-Pipeline erzwingen.
Definition of Done	Checkliste enthaelt Editor-Start, Main.tscn-Start und optional godot --headless --path . --quit.
Checks	Checkliste lesen und einmal ausfuehren, soweit moeglich.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 3. Core-Datenmodell

Step 3.1 - ID- und Enum-Typen

Codex-Aufgabe	Erstelle typed GDScript-Dateien fuer IDs, OwnerType, GameSpeed, ArmyStatus, BattleStatus und EventType.
Nicht tun	Keine Spiellogik.
Definition of Done	Enums/Konstanten sind zentral und werden nicht in UI dupliziert.
Checks	Parser-Check in Godot.

Step 3.2 - CountryData

Codex-Aufgabe	Definiere CountryData als Resource oder Dictionary-Schema mit id, name, ownerId, goldPerMonth, foodPerMonth, defense, center, neighbors.
Nicht tun	Keine echten 180 Laender importieren.
Definition of Done	CountryData-Struktur ist dokumentiert und fuer Fixtures nutzbar.

Checks	Debug-Validierung spaeter anschliessbar.
--------	--

Step 3.3 - UnitData

Codex-Aufgabe	Definiere UnitData fuer infantry, cavalry, artillery mit cost, combatPower, foodUpkeep, moveSpeed und counters/bonuses als optionaler MVP-Wert.
Nicht tun	Keine Panzer, Flugzeuge, Drohnen oder Zeitalter-Einheiten.
Definition of Done	Drei MVP-Einheiten sind modellierbar.
Checks	Godot Script Parser.

Step 3.4 - ArmyData

Codex-Aufgabe	Definiere ArmyData mit id, ownerId, locationCountryId, targetCountryId, units, status, movementProgress.
Nicht tun	Keine Armee-UI und keine Bewegung.
Definition of Done	ArmyData kann von Simulation und UI gemeinsam genutzt werden.
Checks	Parser-Check.

Step 3.5 - RunState

Codex-Aufgabe	Definiere RunState mit time, speed, resources, countries, armies, battles, activeUpgradeChoice, miniGoals, runStatus.
Nicht tun	Keine Save-Serialisierung implementieren.
Definition of Done	RunState bildet MVP-Anforderungen ab und bleibt engine-unabhaengig genug.
Checks	Parser-Check.

Step 3.6 - GameEvent

Codex-Aufgabe	Definiere GameEvent-Struktur fuer countrySelected, armyMoved, battleStarted, battleEnded, countryConquered, upgradeChoiceOpened, missileLaunched.
Nicht tun	Keine Effekte ausloesen.
Definition of Done	Events sind reine Daten und koennen spaeter vom EventBus verteilt werden.
Checks	Parser-Check.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 4. Debug-Testbasis und Datenvalidierung

Step 4.1 - Debug Test Runner

Codex-Aufgabe	Erstelle eine einfache DebugTestRunner-Szene oder Scriptfunktion, die Core-Validierungen ausfuehren kann.
Nicht tun	Kein grosses Testframework installieren.
Definition of Done	Runner kann manuell gestartet werden und gibt klare Pass/Fail-Ausgaben aus.
Checks	DebugTestRunner starten.

Step 4.2 - Country Validator

Codex-Aufgabe	Implementiere Validator fuer CountryData: eindeutige IDs, gueltige Nachbarn, gueltige Owner, Center vorhanden.
Nicht tun	Keine Map rendern.
Definition of Done	Fehlerhafte Fixtures werden klar gemeldet.
Checks	DebugTestRunner.

Step 4.3 - Unit Validator

Codex-Aufgabe	Implementiere Validator fuer UnitData: positive Kosten/Werte, bekannte Unit-IDs, keine negativen Upkeep-Werte.
Nicht tun	Keine Balancing-Diskussion.
Definition of Done	Units sind maschinell pruefbar.
Checks	DebugTestRunner.

Step 4.4 - RunState Validator

Codex-Aufgabe	Implementiere minimale Validierung: Ressourcen nicht NaN, Speed gueltig, Army-Standorte existieren.
Nicht tun	Keine Reparatur-Logik.
Definition of Done	Unguelitger RunState wird erkannt.
Checks	DebugTestRunner.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 5. Prototype Content

Step 5.1 - Unit Fixture

Codex-Aufgabe	Lege data/units.json oder .tres Fixtures fuer infantry, cavalry und artillery an.
Nicht tun	Keine zusaetlichen Einheiten.
Definition of Done	Daten laden und validieren erfolgreich.
Checks	DebugTestRunner.

Step 5.2 - Country Fixture 10-20 Laender

Codex-Aufgabe	Erstelle eine kleine Testkarte mit 10-20 Laendern inklusive Besitzern, Einkommen, Nahrung, Verteidigung, Center.
Nicht tun	Keine echte Weltkarte importieren.
Definition of Done	Ein kompletter Mini-Run kann spaeter auf diesen Laendern stattfinden.
Checks	Country Validator.

Step 5.3 - Adjacency Fixture

Codex-Aufgabe	Fuege Nachbarschaften fuer die Testlaender hinzu.
Nicht tun	Keine globale Pfadsuche.
Definition of Done	Jeder Nachbar verweist auf existierende Laender; Beziehungen sind bidirektional oder bewusst dokumentiert.
Checks	Country Validator.

Step 5.4 - Upgrade Fixture

Codex-Aufgabe	Lege 8-12 einfache passive Run-Upgrades als Daten an.
Nicht tun	Keine aktiven Faehigkeiten.
Definition of Done	Upgrade-Daten sind ladbar und spaeter rollbar.
Checks	DebugTestRunner.

Step 5.5 - MiniGoal Fixture

Codex-Aufgabe	Lege 5-8 einfache Mini-Ziele an: Erobere X Laender, erreiche Gold, besitze Armee, besiege staerkeres Land.
Nicht tun	Keine Story-Questketten.
Definition of Done	Ziele sind als Daten vorhanden und validierbar.
Checks	DebugTestRunner.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 6. GameManager und Run-Erstellung

Step 6.1 - New Run Factory

Codex-Aufgabe	Implementiere create_new_run(startCountryId) auf Basis der Fixtures.
Nicht tun	Keine UI-Auswahl fuer Startland.
Definition of Done	RunState enthaelt Laender, Startressourcen und leere/erste Armeen.
Checks	DebugTestRunner.

Step 6.2 - GameManager State Ownership

Codex-Aufgabe	GameManager haelt den aktuellen RunState und bietet read-only Zugriff fuer UI/Map.
Nicht tun	Keine direkte State-Manipulation durch UI.
Definition of Done	Andere Nodes greifen ueber GameManager/Commands zu.
Checks	Main.tscn starten.

Step 6.3 - Command Skeleton

Codex-Aufgabe	Erstelle Commands fuer select_country, set_game_speed, pause_game, resume_game, reset_run.
Nicht tun	Keine Kampf- oder Wirtschaftscommands.
Definition of Done	Commands aendern State kontrolliert und erzeugen Events, wo sinnvoll.
Checks	DebugTestRunner und Main.tscn.

Step 6.4 - EventBus Basics

Codex-Aufgabe	EventBus kann GameEvents empfangen und Signale weiterleiten.
Nicht tun	Keine Effekte oder Sound anhaengen.
Definition of Done	Country selection Event kann im Output geloggt werden.
Checks	Main.tscn starten.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 7. Simulationstick und Zeitmodell

Step 7.1 - Simulation Clock

Codex-Aufgabe	Implementiere festen Simulationstick mit Accumulator im SimulationManager.
Nicht tun	Keine Wirtschaft oder Bewegung.
Definition of Done	Pause, 1x, 2x, 4x beeinflussen Simulationszeit.
Checks	Debug-Ausgabe und Main.tscn.

Step 7.2 - GameTime

Codex-Aufgabe	Implementiere Wochen/Monate/Jahre gemaess Konzept: 1 Woche = 5 Sekunden bei 1x, 4 Wochen = 1 Monat.
Nicht tun	Keine Monatswirtschaft.
Definition of Done	Zeit laeuft deterministisch und pausierbar.
Checks	DebugTestRunner.

Step 7.3 - Month Tick Event

Codex-Aufgabe	Erzeuge monthTick Events, wenn 4 Wochen vergangen sind.
Nicht tun	Keine Ressourcenberechnung.
Definition of Done	MonthTick kann im Output beobachtet werden.
Checks	Main.tscn laufen lassen.

Step 7.4 - Speed UI Hook Stub

Codex-Aufgabe	Bereite Signale fuer TimeControls vor, ohne UI final zu bauen.
Nicht tun	Keine finalen Buttons.
Definition of Done	SimulationManager reagiert auf set_game_speed.
Checks	Main.tscn.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 8. Szenenstruktur fuer Karte

Step 8.1 - WorldMap Scene

Codex-Aufgabe	Erstelle WorldMap.tscn mit CountryLayer, ArmyLayer, EffectsLayer und MapCamera.
Nicht tun	Noch keine echten Laender rendern.
Definition of Done	Szene kann in Main.tscn instanziiert werden.
Checks	Main.tscn starten.

Step 8.2 - CountryNode Scene

Codex-Aufgabe	Erstelle CountryNode.tscn mit Node2D, Polygon2D, Area2D und CollisionPolygon2D.
Nicht tun	Keine komplexen echten Grenzen.
Definition of Done	CountryNode kann eine vereinfachte Form anzeigen und Klicks empfangen.
Checks	Testszene starten.

Step 8.3 - Map Shape Fixture

Codex-Aufgabe	Ordne jedem Testland einfache Polygonpunkte fuer die Prototypkarte zu.
Nicht tun	Keine echte Weltkarte.
Definition of Done	10-20 Laender haben sichtbare, nicht ueberlappende Testformen.
Checks	WorldMap ansehen.

Step 8.4 - CountryNode Binding

Codex-Aufgabe	WorldMap erzeugt CountryNodes aus CountryData und verbindet Hover/Klick-Signale.
Nicht tun	Keine Kampfkaktionen.
Definition of Done	Klick auf Land sendet select_country Command.
Checks	Main.tscn testen.

Step 8.5 - Owner Coloring

Codex-Aufgabe	CountryNode faerbt sich nach ownerId und zeigt Selection/Hover visuell.
Nicht tun	Keine Diplomatie-Farbgeln.
Definition of Done	Ausgewaehltes Land ist klar erkennbar.

Checks	Main.tscn testen.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 9. Kamera und Karteninput

Step 9.1 - Camera2D Setup

Codex-Aufgabe	Implementiere Camera2D fuer WorldMap mit Startzoom und Begrenzungen.
Nicht tun	Keine Minimap.
Definition of Done	Karte ist bei 1920x1080 gut sichtbar.
Checks	Main.tscn testen.

Step 9.2 - Pan

Codex-Aufgabe	Implementiere Mausziehen oder WASD/Arrow-Pan fuer die Karte.
Nicht tun	Keine Touch-Gesten.
Definition of Done	Pan funktioniert ohne UI-Klicks zu stoeren.
Checks	Manueller Test.

Step 9.3 - Zoom

Codex-Aufgabe	Implementiere Mausrad-Zoom mit min/max Zoom.
Nicht tun	Kein komplexes Smooth-Camera-System.
Definition of Done	Zoom bleibt stabil und zentriert sinnvoll.
Checks	Manueller Test.

Step 9.4 - Input Separation

Codex-Aufgabe	Stelle sicher, dass UI-Control-Klicks nicht versehentlich Map-Commands ausloesen.
Nicht tun	Keine hackigen globalen Input-Abfragen.
Definition of Done	UI und Map koennen getrennt bedient werden.
Checks	Manueller Test.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 10. UI Foundation

Step 10.1 - UIRoot Layout

Codex-Aufgabe	Erstelle UIRoot mit TopBar, LeftPanel, RightPanel, BottomBar und ModalLayer als Control-Nodes.
Nicht tun	Keine finalen Grafiken.
Definition of Done	Layout funktioniert bei 1920x1080.
Checks	Main.tscn testen.

Step 10.2 - TopBar

Codex-Aufgabe	Zeige Gold, Nahrung, Armeestaerke, Bedrohung und Datum aus RunState.
Nicht tun	Keine Berechnung in TopBar.
Definition of Done	TopBar aktualisiert sich durch State/Events.

Checks	Main.tscn testen.
--------	-------------------

Step 10.3 - CountryPanel

Codex-Aufgabe	Zeige ausgewaehltes Land: Name, Besitzer, Gold/Monat, Nahrung/Monat, Verteidigung, stationierte Armeen.
Nicht tun	Keine Angriffsfunktion.
Definition of Done	Panel reagiert auf Land-Auswahl.
Checks	Main.tscn testen.

Step 10.4 - TimeControls

Codex-Aufgabe	Erstelle Pause, 1x, 2x, 4x Buttons, die Commands ausloesen.
Nicht tun	Keine weiteren Settings.
Definition of Done	Buttons beeinflussen SimulationClock.
Checks	Manueller Test.

Step 10.5 - ESC Menu Stub

Codex-Aufgabe	ESC oeffnet ein einfaches Pause-Menue mit Resume und Quit to Menu Stub.
Nicht tun	Keine Save/Load-Buttons.
Definition of Done	ESC pausiert und Resume setzt fort.
Checks	Manueller Test.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 11. Wirtschaftssystem

Step 11.1 - Income Calculation

Codex-Aufgabe	Berechne monatliches Gold und Nahrung aus kontrollierten Laendern.
Nicht tun	Keine Handelsrouten.
Definition of Done	Berechnung ist isoliert und testbar.
Checks	DebugTestRunner.

Step 11.2 - Army Upkeep Calculation

Codex-Aufgabe	Berechne Nahrungskosten aller eigenen Armeen pro Monat.
Nicht tun	Keine komplexe Versorgungslinie.
Definition of Done	Upkeep basiert auf UnitData und ArmyData.
Checks	DebugTestRunner.

Step 11.3 - Month Tick Apply

Codex-Aufgabe	Wende Einkommen und Upkeep bei monthTick auf Ressourcen an.
Nicht tun	Keine UI-Sonderlogik.
Definition of Done	TopBar zeigt veraenderte Werte nach Monatswechsel.
Checks	Main.tscn und DebugTestRunner.

Step 11.4 - Food Shortage MVP

Codex-Aufgabe	Bei 0 Nahrung: keine neue Rekrutierung, keine Heilung, nach kurzer Zeit einfacher Kampfmalus.
Nicht tun	Keine Stabilitaet oder Rebellionen.
Definition of Done	Food-Shortage-Flag/Malus ist im State sichtbar.
Checks	DebugTestRunner.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 12. Armeen und Bewegung

Step 12.1 - Initial Army

Codex-Aufgabe	Erzeuge fuer das Startland eine erste Armee mit Infantry/Cavalry/Artillery.
Nicht tun	Kein Army Editor.
Definition of Done	Armee existiert im RunState und wird im Panel angezeigt.
Checks	DebugTestRunner und Main.tscn.

Step 12.2 - ArmyNode Scene

Codex-Aufgabe	Erstelle ArmyNode.tscn als visuelles Symbol auf der Karte.
Nicht tun	Keine Kampfanimation.
Definition of Done	ArmyNode erscheint am Country Center.
Checks	Main.tscn testen.

Step 12.3 - ArmyLayer Binding

Codex-Aufgabe	ArmyLayer rendert alle Armeen aus RunState und aktualisiert Positionen.
Nicht tun	Keine Pfadsuche.
Definition of Done	Armeen sind auf der Karte sichtbar.
Checks	Main.tscn testen.

Step 12.4 - Move Command

Codex-Aufgabe	Implementiere move_army(armyId, targetCountryId) nur fuer Nachbarlaender.
Nicht tun	Keine globale Route ueber mehrere Laender.
Definition of Done	Ungueltige Bewegungen werden abgelehnt.
Checks	DebugTestRunner.

Step 12.5 - Realtime Movement

Codex-Aufgabe	Armeen bewegen sich sichtbar kontinuierlich zwischen Country Centers, waehrend Simulationsttick den Fortschritt setzt.
Nicht tun	Keine Teleport-Bewegung, keine Physik.
Definition of Done	Bewegung ist pausable und speed-abhaengig.
Checks	Manueller Test.

Step 12.6 - Army Management Panel Stub

Codex-Aufgabe	Erstelle ein einfaches ArmyPanel zum Auswaehlen einer Armee und Anzeigen ihrer Einheiten.
Nicht tun	Keine Split/Merge-Funktion.
Definition of Done	Spieler sieht Standort, Ziel, Status und Einheiten.
Checks	Main.tscn testen.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 13. Rekrutierung und Army Editor MVP

Step 13.1 - Recruit Calculation

Codex-Aufgabe	Berechne Kosten fuer Infantry/Cavalry/Artillery und pruefe Gold/Nahrungslimits.
Nicht tun	Keine Rekrutierungszeit, wenn nicht ausdruecklich noetig.
Definition of Done	Ungueltige Rekrutierung wird abgelehnt.
Checks	DebugTestRunner.

Step 13.2 - Recruit Command

Codex-Aufgabe	Implementiere recruit_units(countryId, unitType, amount).
Nicht tun	Keine komplexen Warteschlangen.
Definition of Done	Gold sinkt, Einheiten steigen, Validierung greift.
Checks	DebugTestRunner.

Step 13.3 - Recruit UI

Codex-Aufgabe	Fuege einfache Rekrutierungsbuttons oder Zahlenfelder im Country/Army Panel hinzu.
Nicht tun	Keine finale UX/Balance.
Definition of Done	Spieler kann MVP-Einheiten rekrutieren.
Checks	Manueller Test.

Step 13.4 - Create Army Command

Codex-Aufgabe	Ermoeegliche eine neue Armee in eigenem Land zu erstellen.
Nicht tun	Keine Merge/Split-Features.
Definition of Done	Neue Armee erscheint im State und auf der Karte.
Checks	DebugTestRunner und Main.tscn.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 14. Kampfsystem und Eroberung

Step 14.1 - Combat Power

Codex-Aufgabe	Berechne Kampfkraft aus Units, UnitData, Food-Malus und einfachen Counter-Boni.
Nicht tun	Kein komplexes Schere-Stein-Papier ueberfrachten.
Definition of Done	Infantry/Cavalry/Artillery werden testbar bewertet.
Checks	DebugTestRunner.

Step 14.2 - Start Attack Command

Codex-Aufgabe	Implementiere start_attack(armyId, targetCountryId) fuer angrenzende feindliche/neutrale Laender.
Nicht tun	Keine Fernangriffe, keine Marine, keine Diplomatie.
Definition of Done	Nur gueltige Angriffe starten BattleState.
Checks	DebugTestRunner.

Step 14.3 - Battle Lifecycle

Codex-Aufgabe	BattleState laeuft automatisch ueber Simulationsttick und endet nach definierter Dauer/Progress.
Nicht tun	Keine taktischen Kampfbuttons.

Definition of Done	Kampf ist pausierbar und deterministisch genug fuer Debug.
Checks	Main.tscn und DebugTestRunner.

Step 14.4 - Casualties

Codex-Aufgabe	Berechne einfache Verluste fuer Gewinner/Verlierer. Units duerfen nicht negativ werden.
Nicht tun	Keine Heilungssysteme.
Definition of Done	Verluste sind nachvollziehbar und testbar.
Checks	DebugTestRunner.

Step 14.5 - Conquest

Codex-Aufgabe	Bei Sieg wechselt ownerId des Ziellands, Armee stationiert dort, countryConquered Event wird erzeugt.
Nicht tun	Keine Stabilitaet oder Besatzungssysteme.
Definition of Done	Besitzerfarbe aendert sich und TopBar/Panels aktualisieren.
Checks	Manueller Test.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 15. Upgrade-System

Step 15.1 - Upgrade Data Validation

Codex-Aufgabe	Validiere Upgrade-Fixtures: id, name, description, rarity, effectType, value.
Nicht tun	Keine aktiven Faehigkeiten.
Definition of Done	Fehlerhafte Upgrade-Daten werden gemeldet.
Checks	DebugTestRunner.

Step 15.2 - Roll 3 Upgrade Choices

Codex-Aufgabe	Nach Eroberung werden 3 zufaellige Upgrades ohne Duplikate ausgewaehlt.
Nicht tun	Keine Rerolls, kein Meta-Shop.
Definition of Done	Genau 3 Karten erscheinen im State.
Checks	DebugTestRunner.

Step 15.3 - Upgrade Modal

Codex-Aufgabe	Zeige UpgradeModal mit 3 Karten, waehrend der Run pausiert oder Interaktion blockiert.
Nicht tun	Keine Shop-UI.
Definition of Done	Spieler kann genau ein Upgrade waehlen.
Checks	Manueller Test.

Step 15.4 - Apply Upgrade Effects

Codex-Aufgabe	Implementiere 5 erste passive Effekte: Rekrutierung billiger, Nahrungskosten runter, Gold aus Eroberung hoch, Bedrohung durch Krieg runter, Defensive Bonus.
Nicht tun	Keine komplexen Build-Synergien.
Definition of Done	Effekte veraendern Berechnungen sichtbar und testbar.
Checks	DebugTestRunner.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 16. Bedrohung und einfache Weltreaktion

Step 16.1 - Passive Threat Growth

Codex-Aufgabe	Bedrohung steigt langsam ueber Zeit.
Nicht tun	Keine Welt-KI.
Definition of Done	Threat steigt bei laufendem Run und bleibt pausiert bei Pause.
Checks	DebugTestRunner und Main.tscn.

Step 16.2 - Threat From Actions

Codex-Aufgabe	Kriege, Eroberungen und sehr grosse Armee erhoehen Bedrohung.
Nicht tun	Keine diplomatischen Konsequenzen.
Definition of Done	Threat-Zuwachs ist zentral berechnet.
Checks	DebugTestRunner.

Step 16.3 - Threat UI States

Codex-Aufgabe	TopBar zeigt Bedrohung mit Warnstufen und ggf. visueller Hervorhebung.
Nicht tun	Keine finalen Art Assets.
Definition of Done	Spieler erkennt Eskalation klar.
Checks	Main.tscn testen.

Step 16.4 - Simple World Reaction Stub

Codex-Aufgabe	Bei hoher Bedrohung erhalten Gegner einfache Staerkeboni oder starten spaeter vorbereitete Gegenangriffe als Stub.
Nicht tun	Keine komplexe Koalitions-KI.
Definition of Done	Stub ist gekapselt und kann spaeter erweitert werden.
Checks	DebugTestRunner.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 17. Mini-Ziele

Step 17.1 - MiniGoal Data Validation

Codex-Aufgabe	Validiere MiniGoal-Fixtures mit Zieltyp, Zielwert, Belohnung und Progress-Regel.
Nicht tun	Keine Questketten.
Definition of Done	Alle Goals sind gueltig.
Checks	DebugTestRunner.

Step 17.2 - MiniGoal Progress

Codex-Aufgabe	Aktualisiere Fortschritt bei Eroberung, Ressourcen, Armee und Kampfergebnissen.
Nicht tun	Keine Story-Events.
Definition of Done	Progress ist im State sichtbar und testbar.
Checks	DebugTestRunner.

Step 17.3 - MiniGoal UI

Codex-Aufgabe	Zeige aktive Mini-Ziele unten links oder in einem Panel.
---------------	--

Nicht tun	Keine finalen Animationen.
Definition of Done	Spieler sieht Fortschritt und abgeschlossene Ziele.
Checks	Main.tscn testen.

Step 17.4 - Rewards

Codex-Aufgabe	Implementiere einfache Belohnungen: Gold, Nahrung, kleine Run-Boni oder seltenere Upgrade-Auswahl.
Nicht tun	Keine permanenten Unlocks.
Definition of Done	Belohnung kann einmal abgeholt werden.
Checks	DebugTestRunner und Main.tscn.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 18. Effekte und Audio

Step 18.1 - EffectsLayer Event Binding

Codex-Aufgabe	EffectsLayer hoert auf GameEvents und startet nur visuelle Effekte.
Nicht tun	Keine Game Logic in EffectsLayer.
Definition of Done	Battle/conquest Events koennen visuell geloggt oder markiert werden.
Checks	Main.tscn testen.

Step 18.2 - Army Movement Feedback

Codex-Aufgabe	Fuege Pfeile, Pfadlinien oder pulsierende Marker fuer Armee-Bewegung hinzu.
Nicht tun	Keine Projekttilphysik.
Definition of Done	Bewegung ist lesbarer ohne Gameplay zu aendern.
Checks	Manueller Test.

Step 18.3 - Battle/Conquest Feedback

Codex-Aufgabe	Land pulsiert bei Kampf, kurzer Flash bei Eroberung.
Nicht tun	Keine grossen Partikeleffekte.
Definition of Done	Feedback raeumt sich nach Ende auf.
Checks	Manueller Test.

Step 18.4 - Missile VFX Prototype

Codex-Aufgabe	Implementiere symbolischen Raketenflug von Startland-Center zu Zielland-Center per Tween.
Nicht tun	Keine Atomraketen-Gameplay-Regeln.
Definition of Done	Rakete fliegt sichtbar und erzeugt Impact-Event/VFX.
Checks	Manueller Test.

Step 18.5 - Explosion VFX

Codex-Aufgabe	Erstelle Explosion mit Partikeln, Flash und optional Screen Shake.
Nicht tun	Keine dauerhaften Performance-Lasten.
Definition of Done	Partikel stoppen und Nodes werden aufgeraeumt.
Checks	Manueller Test und Output pruefen.

Step 18.6 - Audio Buses

Codex-Aufgabe	Erstelle Master, Music, SFX und UI Audio-Busse.
---------------	---

Nicht tun	Keine Musikproduktion.
Definition of Done	Lautstaerken koennen spaeter separat geregelt werden.
Checks	Godot Audio Bus Layout pruefen.

Step 18.7 - Sound Events

Codex-Aufgabe	Verbinde UI-Klick, Kampfstart, Eroberung, Rakete und Explosion mit Sound-Stubs.
Nicht tun	Keine finalen Soundassets erzwingen.
Definition of Done	Platzhalter-Sounds laufen und sind mute-bar.
Checks	Manueller Test.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 19. Save/Load und Meta-Fortschritt

Step 19.1 - Save Schema

Codex-Aufgabe	Definiere versioniertes SaveFormat fuer RunState und MetaProgress.
Nicht tun	Keine Steam Cloud.
Definition of Done	schemaVersion ist enthalten und Migration kann spaeter ergaenzt werden.
Checks	DebugTestRunner.

Step 19.2 - Serialize Run

Codex-Aufgabe	Implementiere Serialisierung ohne Node-Referenzen, Signale oder nicht speicherbare Objekte.
Nicht tun	Keine Binary-Saves.
Definition of Done	JSON speichert reine Daten.
Checks	DebugTestRunner.

Step 19.3 - SaveManager user://

Codex-Aufgabe	Speichere und lade JSON-Dateien unter user://.
Nicht tun	Keine Speicherung im Projektordner.
Definition of Done	Save-Datei wird geschrieben, gelesen und validiert.
Checks	Manueller Test.

Step 19.4 - Manual Save/Load UI

Codex-Aufgabe	Fuege Save/Load Buttons im ESC-Menue hinzu.
Nicht tun	Keine Save-Slot-Galerie.
Definition of Done	Run kann gespeichert und wiederhergestellt werden.
Checks	Manueller Test.

Step 19.5 - MetaProgress Stub

Codex-Aufgabe	Lege Kronen, allgemeine Shop-Upgrades und laenderspezifische Upgrades als MetaProgress-Daten an.
Nicht tun	Kein voller Shop, falls nicht bereit.
Definition of Done	Meta-Daten koennen gespeichert und geladen werden.
Checks	DebugTestRunner.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten

Phase fortfahren.

Phase 20. Shop und Meta-Progress MVP

Step 20.1 - Crowns Reward

Codex-Aufgabe	Berechne Kronen nach Run-Ende auf Basis einfacher Leistungswerte.
Nicht tun	Keine Pay-to-win-Mechanik.
Definition of Done	Run-Ende erzeugt nachvollziehbare Kronen.
Checks	DebugTestRunner.

Step 20.2 - Shop Data

Codex-Aufgabe	Erstelle erste allgemeine und laenderspezifische Shop-Upgrades als Daten.
Nicht tun	Keine Laender-Unlocks.
Definition of Done	Shop-Daten sind validiert.
Checks	DebugTestRunner.

Step 20.3 - Shop UI Stub

Codex-Aufgabe	Erstelle ein einfaches Shop-Menue im Hauptmenue.
Nicht tun	Keine finale Shop-Grafik.
Definition of Done	Spieler sieht Kronen und Upgrades.
Checks	Manueller Test.

Step 20.4 - Purchase Upgrade

Codex-Aufgabe	Implementiere Kauf, Kostenpruefung und Persistenz im MetaProgress.
Nicht tun	Keine Refunds, wenn nicht benoetigt.
Definition of Done	Gekaufte Upgrades bleiben nach Neustart erhalten.
Checks	DebugTestRunner und manueller Test.

Step 20.5 - Apply Starting Bonuses

Codex-Aufgabe	Wende gekaufte Startboni beim New Run an.
Nicht tun	Keine massiven Schadensboni, die Balancing zerstoen.
Definition of Done	Startressourcen/Laenderboni aendern sich sichtbar.
Checks	DebugTestRunner.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 21. Desktop-Einstellungen und Bedienung

Step 21.1 - Settings Data

Codex-Aufgabe	Definiere Settings fuer masterVolume, musicVolume, sfxVolume, uiScale, fullscreen/windowed.
Nicht tun	Keine Grafikoptionen-Flut.
Definition of Done	Settings sind versioniert und speicherbar.
Checks	DebugTestRunner.

Step 21.2 - Settings UI

Codex-Aufgabe	Erstelle Settings-Menue mit Audio und UI Scale.
Nicht tun	Keine finalen Keybinds.
Definition of Done	Aenderungen wirken sofort und werden gespeichert.

Checks	Manueller Test.
--------	-----------------

Step 21.3 - Fullscreen/Windowed

Codex-Aufgabe	Implementiere Umschalten zwischen Fullscreen und Windowed ueber Godot DisplayServer.
Nicht tun	Keine Multi-Monitor-Spezialfaelle.
Definition of Done	Toggle funktioniert im lokalen Build/Editor.
Checks	Manueller Test.

Step 21.4 - Input Map

Codex-Aufgabe	Definiere Aktionen fuer Pause, Pan, Zoom, ESC, optional Speed Hotkeys.
Nicht tun	Kein Controller-Support im MVP.
Definition of Done	InputMap ist zentral und dokumentiert.
Checks	Manueller Test.

Step 21.5 - Error Overlay Debug

Codex-Aufgabe	Erstelle nur fuer Debug ein einfaches Overlay oder Logging fuer kritische Fehler.
Nicht tun	Keine Telemetrie.
Definition of Done	Fehler sind beim Testen schneller sichtbar.
Checks	Main.tscn testen.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 22. Steam-Vorbereitung ohne harte Integration

Step 22.1 - PlatformService Interface

Codex-Aufgabe	Definiere PlatformService mit Methoden fuer achievements, stats, cloudReady, userName als Mock.
Nicht tun	Keine GodotSteam-Abhaengigkeit.
Definition of Done	Game Code haengt nur am Interface.
Checks	DebugTestRunner.

Step 22.2 - Achievement Event Map

Codex-Aufgabe	Mappe wichtige GameEvents auf interne Achievement-IDs.
Nicht tun	Keine echten Steam-Achievements freischalten.
Definition of Done	Achievement-Events sind zentral dokumentiert.
Checks	DebugTestRunner.

Step 22.3 - Steam Build Notes

Codex-Aufgabe	Erstelle docs/steam_build_notes.md mit spaeterem GodotSteam/SteamPipe-Vorgehen und offenen Voraussetzungen.
Nicht tun	Keine Store-Seite oder rechtliche Angaben erfinden.
Definition of Done	Dokument beschreibt, was spaeter noch zu tun ist.
Checks	Manuelle Pruefung.

Step 22.4 - GodotSteam Decision Gate

Codex-Aufgabe	Lege ein ADR-Dokument an: Wann GodotSteam eingebaut wird und was vorher stabil sein muss.
---------------	---

Nicht tun	GodotSteam noch nicht installieren.
Definition of Done	Entscheidung ist vorbereitet, aber MVP bleibt SDK-frei.
Checks	Manuelle Pruefung.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 23. Export und Packaging

Step 23.1 - Windows Export Preset

Codex-Aufgabe	Richte Windows Desktop Export Preset ein.
Nicht tun	Noch kein SteamPipe-Upload.
Definition of Done	Export-Preset existiert und nutzt sinnvolle Dateinamen.
Checks	Godot Export Dialog pruefen.

Step 23.2 - Icon und App Metadata Platzhalter

Codex-Aufgabe	Setze Projektname, Version und Platzhalter-Icon.
Nicht tun	Keine finalen Store Assets.
Definition of Done	Export zeigt Paper Empire korrekt an.
Checks	Lokaler Export.

Step 23.3 - Production Smoke Test

Codex-Aufgabe	Exportiere einen Windows-Testbuild und pruefe Start, New Run, Map, Movement, Save/Load, Quit.
Nicht tun	Keine neuen Features waehrend Smoke Test.
Definition of Done	Checkliste wird bestanden oder Blocker dokumentiert.
Checks	Manueller Build-Test.

Step 23.4 - Performance Snapshot

Codex-Aufgabe	Pruefe FPS und CPU/GPU grob im Vertical-Slice-Build.
Nicht tun	Keine Optimierung ohne gemessenen Grund.
Definition of Done	Offensichtliche Performance-Probleme sind dokumentiert.
Checks	Godot Profiler grob pruefen.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

Phase 24. Vertical Slice Gate

Step 24.1 - Mini-Run Komplett

Codex-Aufgabe	Stelle sicher, dass ein Mini-Run mit 10-20 Laendern, 3 Einheiten, Wirtschaft, Angriff, Eroberung und Upgrade funktioniert.
Nicht tun	Keine neuen Systeme.
Definition of Done	Ein kompletter Mini-Run ist spielbar.
Checks	Manueller Run + DebugTestRunner.

Step 24.2 - Bugfix Freeze

Codex-Aufgabe	Behebe nur Blocker und klare Bugs aus dem Mini-Run.
Nicht tun	Keine Feature-Arbeit.
Definition of Done	Bekannte Blocker sind geschlossen oder dokumentiert.
Checks	Manueller Test.

Step 24.3 - Balance Pass 1

Codex-Aufgabe	Passe Werte minimal an, damit der Mini-Run weder trivial noch unmöglich ist.
Nicht tun	Kein Rework des Kampfsystems.
Definition of Done	Werte stehen in Daten, nicht hart in UI-Scripts.
Checks	Manueller Test.

Step 24.4 - Architecture Review

Codex-Aufgabe	Pruefe, ob Godot-Struktur, Szenen, Scripts und Daten weiterhin sauber getrennt sind.
Nicht tun	Kein spontaner Engine-Wechsel.
Definition of Done	Kurzes ADR/Review-Dokument liegt vor.
Checks	Manuelle Pruefung.

Gate nach dieser Phase

Projekt starten, Output pruefen, relevante Debug-Checks ausfuehren und Git-Commit setzen. Danach erst mit der naechsten Phase fortfahren.

6. Review-Prompt fuer Codex

Review Step X.Y implementation only.
Check for these risks:

- Game logic inside UI scripts.
- Visual effects changing gameplay state.
- Unexpected .tscn changes or renamed nodes.
- Untyped or overly broad GDScript.
- Steam, web, mobile or future MVP systems added too early.

Do not add features during the review.
Return findings, changed files and suggested fixes only.

7. Anti-Scope-Creep Liste

- Keine Webversion.
- Keine Electron/React/Vite/PixiJS-Architektur.
- Keine Mobile-App.
- Keine echte Steamworks-Integration vor Vertical Slice.
- Keine Diplomatie.
- Keine Marine.
- Kein Handel.
- Keine Stabilitaet/Rebellionen.
- Keine Provinzen.
- Keine Forschung.
- Kein Multiplayer.
- Keine Account-/Backend-Systeme.
- Keine komplette Weltkarte importieren, bevor der Prototyp mit Testlaendern funktioniert.